

Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Сибирский государственный университет телекоммуникаций и
информатики»
(СибГУТИ)

Кафедра ТСиВС

ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ

**По дисциплине: «Интернет-технологии и сетевое
программное обеспечение»**

Тема: «Сервис регистрации и обработки фотографий»

Студент группы ИКС-433

ФИО студента Добромилов Артём

Проверил:

Должность

ФИО

Новосибирск 2026 г.

Оглавление

Введение	3
1 Постановка задачи	4
1.1 Назначение системы	4
1.2 Функциональные требования	4
1.3 Нефункциональные требования	5
1.4 Соответствие варианту задания	5
2 Архитектура приложения	6
2.1 Состав компонентов	6
2.2 Выбор протоколов	7
2.3 Хранение и инициализация данных	7
2.3.1 Порядок запуска хранилищ	9
3 Реализация приложения	10
3.1 Сервис фотографий	10
3.2 Сервис обработки	10
3.3 API-шлюз	11
3.4 Обработка ошибок	12
4 Развертывание и проверка	13
4.1 Контейнеризация	13
4.2 Проверка сценария работы	13
4.3 Автоматизация сборки и развертывание	14
4.4 Ограничения реализации	15
Заключение	16
Список использованных источников	17

Введение

Современные веб-приложения часто работают с пользовательскими медиафайлами: сохраняют сведения о них, запускают фоновые операции и отображают результат обработки через внешний программный интерфейс. Разделение этих обязанностей между сервисами позволяет независимо изменять пользовательское API, правила хранения и длительные фоновые операции.

В работе разработана учебная распределенная система `photos-s06` на языке Go. Пользователь регистрирует фотографию по URL, после чего система создает задачу обработки и предоставляет текущий статус. Обработка моделируется переходом состояний `pending` -> `processing` -> `completed`; скачивание и изменение бинарного изображения не выполняются.

Цель работы -- разработать контейнеризованное приложение, демонстрирующее внешнее REST/GraphQL API, внутреннее gRPC-взаимодействие сервисов и раздельное хранение данных в PostgreSQL.

Для достижения цели решаются следующие задачи:

- определить состав сервисов и их зоны ответственности;
- разработать gRPC-контракты для ресурса фотографии и задачи обработки;
- реализовать HTTP-шлюз с REST и GraphQL интерфейсами;
- организовать хранение данных в отдельных базах PostgreSQL;
- обеспечить запуск приложения посредством Docker Compose;
- предусмотреть автоматическую проверку сборки и тестов.

При разработке используются язык Go [1], технология gRPC [2], СУБД PostgreSQL [3] и контейнеризация Docker [4].

1 Постановка задачи

1.1 Назначение системы

Разрабатываемое приложение предназначено для регистрации фотографий, представленных абсолютными HTTP/HTTPS-ссылками, и наблюдения за процессом их обработки. Пользователь взаимодействует только со шлюзом `api-gateway`; внутренние сервисы скрыты за gRPC-контрактами.

Основной ресурс системы имеет вид, приведенный в листинге 1.1.

Листинг 1.1 – Представление фотографии во внешнем API

```
{
  "id": "photo_...",
  "url": "https://example.com/image.jpg",
  "created_at": "2026-05-26T10:00:00Z",
  "processing_status": "pending"
}
```

1.2 Функциональные требования

Система должна обеспечивать следующие функции:

- 1) создание ресурса **Photo** по валидному абсолютному URL;
- 2) выдачу списка сохраненных фотографий и одной фотографии по идентификатору;
- 3) автоматическую постановку новой фотографии на обработку;
- 4) выдачу состояния обработки и повторный запуск постановки задачи;
- 5) доступ к основным операциям через REST и GraphQL;
- 6) хранение фотографий и заданий обработки в разных базах данных.

Во внешнем API ресурс соответствует варианту `photos-s06`: основное поле ресурса -- `url`, gRPC-сервис -- `PhotosService`, а GraphQL-операции -- `photos` и `createPhoto`.

1.3 Нефункциональные требования

Для воспроизводимости запуска сервисы должны собираться в Docker-контейнеры и запускаться одной командой Docker Compose. Граница взаимодействия между сервисами описывается файлами Protocol Buffers, что уменьшает вероятность несогласованных изменений клиента и сервера.

Приложение должно корректно обрабатывать ошибочные входные данные: некорректный URL должен завершаться ответом **400 Bad Request**, а обращение к неизвестной фотографии -- ответом **404 Not Found**. Доступность компонентов контролируется health-эндпоинтами.

1.4 Соответствие варианту задания

Основные параметры варианта и их реализация в разработанном приложении приведены в таблице 1.1.

Таблица 1.1 – Реализация параметров варианта `photos-s06`

Параметр варианта	Реализация
<code>resource: photos</code>	Сущность <code>Photo</code> , маршрут <code>/api/photos</code>
<code>extra_field: url</code>	Обязательная HTTP/HTTPS-ссылка фотографии
<code>photos-svc-s06:8235</code>	gRPC-сервис хранения фотографий
<code>photos.v1.PhotosService</code>	Контракт в файле <code>proto/photo.proto</code>
<code>photos.createPhoto</code>	GraphQL-запрос и мутация внешнего шлюза

2 Архитектура приложения

2.1 Состав компонентов

Система состоит из трех прикладных сервисов и двух баз данных. Назначение компонентов приведено в таблице 2.1.

Таблица 2.1 – Компоненты системы

Компонент	Назначение
api-gateway	Внешний REST и GraphQL API, объединение данных фотографии со статусом обработки
photos-svc-s06	Создание и чтение фотографий, запуск обработки; gRPC-порт 8235
processing-svc	Создание задач и изменение их статусов; gRPC-порт 8236
photos-db	PostgreSQL-база ресурса Photo
processing-db	PostgreSQL-база фоновых заданий обработки

Общая схема взаимодействия представлена на рисунке 2.1.

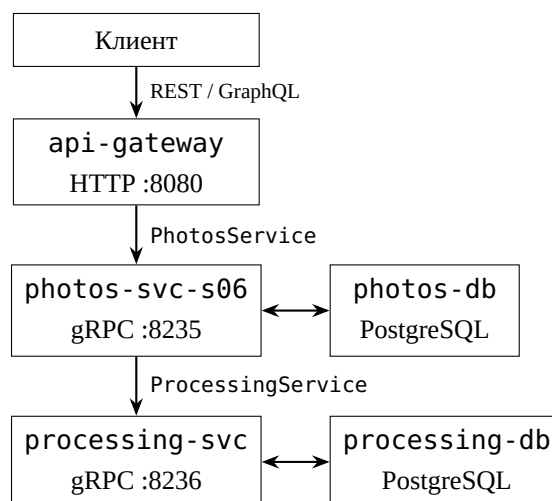


Рисунок 2.1 – Архитектура сервиса обработки фотографий

2.2 Выбор протоколов

Внешний шлюз принимает HTTP/JSON-запросы. REST API удобно применять при ручной проверке с помощью `curl` и при создании обычного клиентского приложения. GraphQL предоставляет альтернативный интерфейс, в котором клиент явно указывает необходимые поля ответа [5].

Внутренние обращения выполняются по gRPC. Контракты определены в `proto/photo.proto` и `proto/processing.proto`, а исходный код клиентов и серверов генерируется для Go. Данный подход предоставляет типизированные сообщения и явно описывает методы межсервисного API.

Таблица 2.2 – Внешние HTTP-маршруты шлюза

Метод	Маршрут	Назначение
POST	<code>/api/photos</code>	Создание фотографии
GET	<code>/api/photos</code>	Список фотографий
GET	<code>/api/photos/{id}</code>	Чтение фотографии
POST	<code>/api/photos/{id}/process</code>	Повторный запуск обработки
POST	<code>/graphql</code>	GraphQL-запросы и мутации

2.3 Хранение и инициализация данных

Каждый бизнес-сервис владеет собственной базой данных. Таблица `photos` содержит идентификатор, URL и время создания фотографии. Таблица `processing_jobs` содержит задачу обработки, ссылку на фотографию, статус и временные метки. Прямое чтение чужой таблицы между сервисами отсутствует: обмен выполняется по gRPC.

Разделение данных уменьшает связанность компонентов и позволяет независимо изменять логику обработки.

Создание данных выполняется в два этапа. Контейнеры PostgreSQL при первом запуске создают логические базы `photos` и `processing` по переменным `POSTGRES_DB` из файла `docker-compose.yml`. После установки

соединения Go-сервисы самостоятельно создают принадлежащие им таблицы. Соответствие операций исходным файлам показано в таблице 2.3.

Таблица 2.3 – Инициализация хранилищ приложения

Файл	Выполняемая инициализация
<code>docker-compose.yml</code>	Создание баз <code>photos</code> и <code>processing</code> переменными PostgreSQL <code>POSTGRES_DB</code>
<code>internal/photos/service.go</code>	Создание таблицы <code>photos</code> запросом <code>CREATE TABLE IF NOT EXISTS</code> в функции <code>New</code>
<code>internal/processing/service.go</code>	Создание таблицы <code>processing_jobs</code> запросом <code>CREATE TABLE IF NOT EXISTS</code> в функции <code>New</code>

Листинг 2.1 – Создание логических баз при старте PostgreSQL

```
photos-db:
  environment:
    POSTGRES_DB: photos
processing-db:
  environment:
    POSTGRES_DB: processing
```

Подключение к базе передается сервисам через `DATABASE_URL`. Использование `IF NOT EXISTS` делает повторный запуск контейнеров безопасным для уже созданных таблиц. DDL-запросы, выполняемые функциями `photos.New` и `processing.New`, представлены в листинге 2.2.

Листинг 2.2 – Создание таблиц прикладными сервисами

```
CREATE TABLE IF NOT EXISTS photos (  
  id TEXT PRIMARY KEY,  
  url TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS processing_jobs (  
  id TEXT PRIMARY KEY,  
  photo_id TEXT NOT NULL UNIQUE,  
  photo_url TEXT NOT NULL,  
  status TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL,  
  updated_at TIMESTAMPTZ NOT NULL  
);
```

Для рабочей среды вместо встроенной инициализации следует применять версионированные миграции схемы.

2.3.1 Порядок запуска хранилищ

При выполнении команды `docker compose up` запуск выполняется последовательно: сначала PostgreSQL инициализирует базы в именованных томах, затем health-check подтверждает готовность соединений, после чего стартуют `processing-svc` и `photos-svc-s06`. Вызов функций `New` создает отсутствующие таблицы до начала обработки запросов. Шлюз запускается последним, когда оба внутренних сервиса уже доступны.

Именованные тома `photos-data` и `processing-data` сохраняют содержимое баз после остановки контейнеров. Полное удаление тестовых данных выполняется только при явном запуске `docker compose down -v`.

3 Реализация приложения

3.1 Сервис фотографий

Сервис `photos-svc-s06` реализован в каталоге `internal/photos`. При создании фотографии проверяется корректность URL: разрешены только абсолютные адреса со схемами `http` и `https`. Идентификатор формируется с применением криптографического генератора случайных данных. Сохранение выполняется параметризованным SQL-запросом.

После успешной записи фотографии сервис вызывает метод `StartProcessing`. Если сервис обработки временно недоступен, фотография остается сохраненной, а пользователь может повторно поставить ее на обработку через шлюз.

Листинг 3.1 – Проверка адреса фотографии

```
func ValidateURL(rawURL string) error {
    parsed, err := url.ParseRequestURI(rawURL)
    if err != nil || parsed.Host == "" ||
        (parsed.Scheme != "http" && parsed.Scheme != "https") {
        return fmt.Errorf("url must be an absolute http or https URL")
    }
    return nil
}
```

3.2 Сервис обработки

Сервис `processing-svc` принимает задание с идентификатором фотографии и ее URL. В базе фиксируется задача со статусом `pending`. Фоновый `worker` периодически обновляет состояние: `pending` переходит в `processing`, затем в `completed`. Такая модель позволяет показать асинхронную операцию без избыточной реализации обработки изображений.

Таблица 3.1 – Жизненный цикл задания обработки

Статус	Условие перехода	Смысл состояния
pending	Создание задания	Задание принято сервисом
processing	Истекла половина за- держки	Обработка выполняется
completed	Истекла полная за- держка	Обработка завершена

Листинг 3.2 – Переход задания в завершающее состояние

```
UPDATE processing_jobs
SET status = 'completed', updated_at = $1
WHERE status = 'processing' AND updated_at <= $2;
```

3.3 API-шлюз

Шлюз не содержит собственной базы данных. При REST- или GraphQL-запросе он обращается к сервису фотографий, затем запрашивает статус у сервиса обработки и строит единое клиентское представление. Если статус временно нельзя получить, пользователь получает фотографию со значением `unavailable`, а не ошибку потери основного ресурса.

Пример создания фотографии через REST приведен в листинге 3.3.

Листинг 3.3 – Создание фотографии через REST API

```
curl -X POST http://localhost:8080/api/photos \
-H "Content-Type: application/json" \
-d '{"url":"https://example.com/photo.jpg"}'
```

GraphQL-мутация, выполняющая аналогичную операцию, приведена в листинге 3.4.

Листинг 3.4 – Создание фотографии через GraphQL API

```
mutation {
  createPhoto(url: "https://example.com/photo.jpg") {
    id url processingStatus
  }
}
```

```
}
```

3.4 Обработка ошибок

Обработчик JSON внешнего API отклоняет неизвестные поля. Ошибки gRPC преобразуются в соответствующие HTTP-статусы: некорректное значение URL становится ответом **400**, неизвестный ресурс -- **404**, а недоступность внутреннего сервиса -- **503**. Health-эндпоинты проверяют соединение с базами данных и используются Docker Compose для упорядоченного старта компонентов.

4 Развертывание и проверка

4.1 Контейнеризация

Прикладные сервисы собираются одним многостадийным `Dockerfile`. На стадии сборки используется Go-образ, а в результирующий Alpine-образ копируется только скомпилированный бинарный файл. Запуск выполняется от непривилегированного пользователя `app`.

Файл `docker-compose.yml` описывает пять контейнеров: два экземпляра PostgreSQL, два gRPC-сервиса и HTTP-шлюз. Для баз используются именованные тома, поэтому сохраненные данные переживают перезапуск контейнеров.

Команда запуска приложения приведена в листинге 4.1.

Листинг 4.1 – Локальный запуск системы

```
docker compose up --build
```

4.2 Проверка сценария работы

После запуска `gateway` публикуется на порту 8080. Последовательность ручной проверки приведена в таблице 4.1.

Таблица 4.1 – Проверка основного сценария

Шаг	Действие	Ожидаемый результат
1	POST /api/photos с корректным URL	Создан ресурс со статусом pending
2	GET /api/photos	Фото присутствует в списке
3	Повторный запрос после задержки обработки	Статус равен completed
4	Создание с относительным URL	Ответ 400 Bad Request
5	Запрос несуществующего идентификатора	Ответ 404 Not Found

Для проверки функции валидации URL реализованы модульные тесты. Они проверяют допустимые HTTP/HTTPS-адреса, пустое значение, относительный адрес и неподдерживаемую схему `ftp`. Тесты запускаются командой:

Листинг 4.2 – Запуск модульных тестов

```
go test ./...
```

4.3 Автоматизация сборки и развертывание

Workflow GitHub Actions запускает тесты Go, проверяет корректность Docker Compose-конфигурации и собирает контейнеры при отправке изменений и при создании pull request. Такой конвейер выявляет ошибки сборки до передачи проекта на проверку.

Таблица 4.2 – Этапы автоматической проверки

Этап	Выполняемая команда и результат
Модульные тесты	<code>go test ./...</code> : проверка валидации URL и сборки пакетов
Валидация Compose	<code>docker compose config --quiet</code> : проверка конфигурации сервисов
Сборка образов	<code>docker compose build</code> : сборка трех исполняемых сервисов

В каталоге `k8s` находятся манифесты Kubernetes: `Deployment` и `Service` для компонентов, `Secret` для строк подключения к базам данных и `Ingress` для HTTP-шлюза. Данные файлы демонстрируют возможный вариант развертывания; перед использованием вне учебной среды пароли должны передаваться из защищенного хранилища, а образы -- публиковаться в `container registry`.

4.4 Ограничения реализации

Учебная система не скачивает изображение по URL и не выполняет реальных преобразований фотографии. Также в проекте отсутствуют пользовательская аутентификация и TLS-терминация. При дальнейшем развитии следует добавить авторизацию, HTTPS на входном шлюзе, миграции базы данных и хранение загруженных объектов в специализированном файловом хранилище.

Заключение

В результате работы разработан проект `photos - s06`, реализующий регистрацию фотографий и отслеживание асинхронной обработки. Приложение содержит внешний HTTP-шлюз с REST и GraphQL API, два взаимодействующих по gRPC бизнес-сервиса и две изолированные базы PostgreSQL.

В проекте реализованы проверка URL, обработка ошибок межсервисного взаимодействия, health-эндпоинты, контейнерный запуск и автоматическая проверка тестов и сборки. Предложенная структура демонстрирует основные принципы построения распределенного приложения: явные контракты, разделение ответственности сервисов и воспроизводимое развертывание.

Дальнейшее развитие проекта может включать загрузку исходного файла, реальную обработку изображения, защищенный доступ пользователей, TLS и версионированные миграции данных.

Список использованных источников

1. The Go Programming Language Documentation. — URL: <https://go.dev/doc/> (дата обр. 26.05.2026).
2. gRPC Documentation. — URL: <https://grpc.io/docs/> (дата обр. 26.05.2026).
3. PostgreSQL Documentation. — URL: <https://www.postgresql.org/docs/> (дата обр. 26.05.2026).
4. Docker Documentation. — URL: <https://docs.docker.com/> (дата обр. 26.05.2026).
5. GraphQL Documentation. — URL: <https://graphql.org/learn/> (дата обр. 26.05.2026).